VCL Motion Matching Project Report

王唐欣宇

January 10, 2026

1 Introduction

VCL lectures covered basic animation concepts, including forward kinematics and inverse kinematics. However, advanced animation techniques were outside the scope of the standard curriculum. Therefore, for this final project, I implemented several advanced animation techniques from the Animation subtopics, including specific BVH Motion Player, Skinning, and Motion Matching.

The BVH Motion Player is a direct application of forward kinematics, allowing the loading and playback of motion capture data stored in BVH files.

Skinning is a technique used to deform a character's mesh based on the underlying skeleton, enabling more realistic character animations. I used Linear Blend Skinning (LBS) to achieve this effect.

Motion Matching is an advanced animation technique that allows for real-time selection and blending of motion capture data based on the character's current state and desired movement. I built a motion matching system on top of the BVH Motion Player and Skinning implementations.

In this project, I integrated all selected topics—BVH Motion Player, Skinning, and Motion Matching—to form a complete character animation system.

2 Project Overview

The project codebase is organized into a modular structure that separates assets, core logic, and application functionality. The primary directory organization is as follows:

- **Assets and Data Resources** (`assets/`): Contains all non-code resources.
  - `bvh/`: Motion capture data in BVH format.
  - `data/`: 3D character models and pre-computed motion databases.
  - `shaders/`: GLSL shader files character skinning and scene rendering.
- **Core Logic** (`src/VCX/Labs/MotionMatching/Core/`): The heart of the animation system.
  - `Math`: Fundamental data structures (vectors, quaternions) and mathematical utilities.
  - `Animation`: Implementations for skeleton hierarchy, forward/inverse kinematics, and the motion matching search engine.
- **Application & Rendering** (`src/VCX/Labs/MotionMatching/`):
  - `SimulationObject*` & `SceneEnvironment*`: Handling the high-level rendering loop.
  - `Case*`: The entry points for the three demonstration cases (BVH Player, Skinning, Motion Matching).

3 Implementation Details

To ensure maintainability and extensibility, I decoupled the architecture into nine distinct functional components. These components interact to form the complete motion matching pipeline.

1. **Animation Bones Operation**: Manages the skeletal hierarchy, forward and inverse kinematics, and local-to-global bone transformations, ensuring correct anatomical structure and movement.
2. **Rendering**: Manages OpenGL draw calls, shader programs, and vertex buffers.
3. **GUI**: Provides an interactive visual interface for debugging and real-time parameter tuning using ImGui.
4. **Skinning**: Implements Linear Blend Skinning (LBS) to deform the character mesh based on the skeletal poses computed by the animation system.
5. **Matching Database**: The core engine responsible for storing motion data, extracting features, and executing efficient nearest-neighbor searches to find the best matching animation frames.
6. **Pose Update**: The central logic hub ('MotionMatchingUpdate') that orchestrates the frame update. It utilizes the **Controller** to predict the future trajectory, updates the simulation object's state (with smoothing), performs the database search, and computes the final character pose.
7. **Controller**: Provides the trajectory prediction logic and computes desired velocities/directions based on input. It is called by the Pose Update component.
8. **Whole Processing**: The top-level application cycle ('CaseMotionMatching'). It manages GUI state and input capture, then sequentially executes the Pose Update, Skinning, and Rendering phases for each frame.
9. **Utils**: A collection of foundational mathematical libraries and data structures used throughout the system.

The following subsections detail the implementation of these components across the three demonstration cases: BVH Motion Player, Skinned BVH Motion Player, and Motion Matching System.

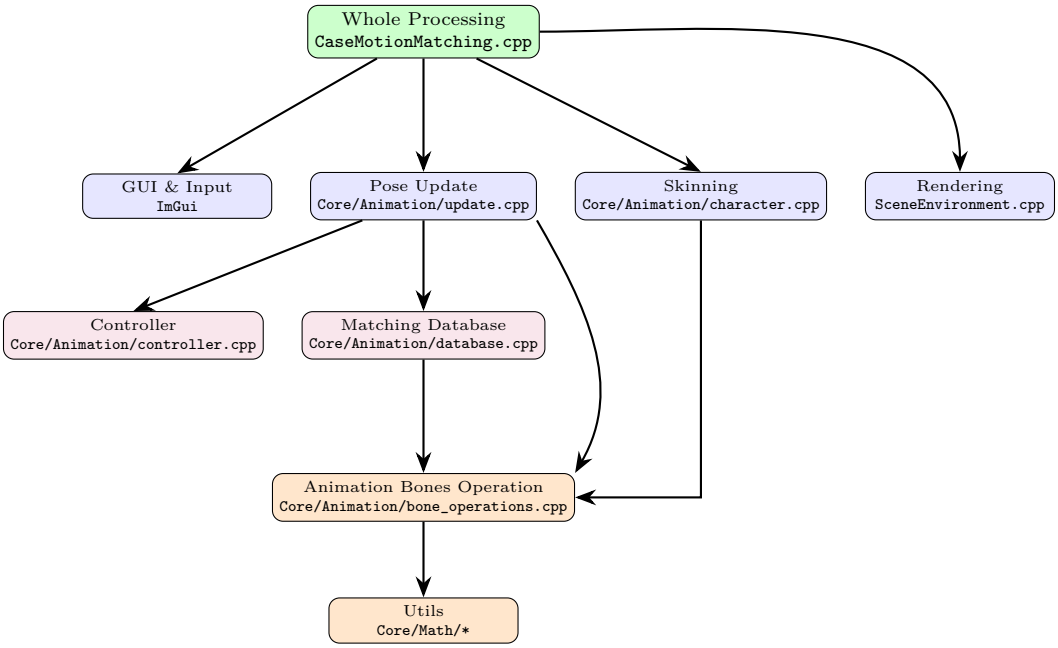


Figure 1: System Architecture and File Dependencies

3.1 BVH Motion Player

This case utilizes components 1. Animation Bones Operation, 2. Rendering, 3. GUI, and 9. Utils to load and play BVH motion capture data.

I used a third-party BVH parser to read the BVH files and extract the skeleton hierarchy and motion data, located in `bvh11.h` and `bvh11.cpp`.

In the Animation Bones Operation component, I implemented the `forward_kinematics_full` function to compute the global transformation of each bone based on its local transformation and its parent's global transformation. The update formula is:

$$G_i = G_{p(i)} \cdot L_i$$

where G_i is the global transformation of bone i, $G_{p(i)}$ is the global transformation of its parent bone, and L_i is the local transformation of bone i. I implemented this using an iterative loop rather than recursion. Since the bones are stored in a topological order (where a parent always has a lower index than its child), I can compute global transformations in a single linear pass from index 0 to N.

For character rendering, I used a simple shader program `skeleton.vert` and `skeleton.frag` to visualize the skeleton structure. Specifically, I represented each bone by rendering a transformed cuboid (scaled unit cube). For each bone connecting a joint to its parent, I calculated a model matrix that scales the cube to the bone's length, rotates it to align with the bone's direction, and translates it to the midpoint between the two joints.

For environment rendering, I used another shader program `checkerboard.vert` and `checkerboard.frag` to render a ground plane for better visual context during animation playback, with the base plane textured with a checkerboard pattern. The XYZ axes are also rendered in the `SceneEnvironment*` class for better orientation. This rendering class is reused in the subsequent cases.

3.2 Skinned BVH Motion Player

This case builds upon the BVH Motion Player, additionally utilizing component 4. Skinning to apply Linear Blend Skinning (LBS) for realistic character mesh deformation during animation playback.

The bone operations and scene environment rendering remain consistent with the BVH Motion Player case.

For skinning, I implemented the Linear Blend Skinning (LBS) algorithm to deform the character mesh. I first load mesh data and bone weights from `Character.bin`. For each vertex $\mathbf{v}$, its deformed position $\mathbf{v}'$ is calculated as:

$$\mathbf{v}' = \sum_i w_i \left(\mathbf{q}_i \cdot (\mathbf{q}_{rest,i}^{-1}(\mathbf{v} - \mathbf{t}_{rest,i})) + \mathbf{t}_i \right)$$

where w_i is the weight associated with bone i, $(\mathbf{q}_{rest,i}, \mathbf{t}_{rest,i})$ denotes the bind pose transformation, and $(\mathbf{q}_i, \mathbf{t}_i)$ is the animated global transformation.

A **retargeting** step is required because the character's skeletal structure (from `Character.bin`) includes a special root bone (Index 0), known as the Simulation Bone, which helps decouple the character's logical position from its animation pose. The BVH motion data, however, starts at the Hips. Therefore, I mapped the BVH joints to the skinning skeleton starting from Index 1. The global position $\mathbf{P}_{sim}$ of the Simulation Bone is derived by projecting the position of a reference bone (e.g., Spine2) onto the ground plane:

$$\mathbf{P}_{sim} = \begin{bmatrix} P_{spine}.x, & 0, & P_{spine}.z \end{bmatrix}^T$$

In the GPU rendering stage, I used a Half-Lambert shading formula in the fragment shader: $I = (\mathbf{N} \cdot \mathbf{L} + 1)/2$, which preserves details in low-light regions.

3.3 Motion Matching System

This case extends the Skinned BVH Motion Player, incorporating components 5. Matching Database, 6. Pose Update and 7. Controller to implement a real-time motion matching system. It allows for dynamic selection and blending of motion capture data based on user input and character state.

The **Whole Processing** component orchestrates the execution flow. In each frame, it first captures user input via the **GUI**. The core animation logic is then delegated to the **Pose Update** component, which internally utilizes the **Controller** for trajectory prediction and the **Matching Database** for frame selection to determine the optimal character pose. Once the pose is computed, the **Skinning** component performs mesh deformation, and finally, the **Rendering** component visualizes the scene.

3.3.1 Input Processing and Simulation Object

The system distinguishes between two locomotion modes: **Walking** and **Running**, which are controlled by user input (e.g., holding the Shift key). Additionally, a **Strafing** mode is supported, allowing the character to move laterally while maintaining a fixed facing direction.

The user's raw input (WASD keys) is first processed to handle deadzones and normalization, then transformed into a world-space *desired velocity* that accounts for the camera's orientation. Anisotropic speed limits are applied to allow different maximum speeds for forward, backward, and sideways movement.

Rather than applying the desired velocity instantaneously, a **Simulation Object** is used to smooth the character's motion. This object acts as a filter, gradually adjusting the character's position and velocity towards the desired velocity, simulating inertia and producing a smooth, physically plausible trajectory. The predicted future states generated by this system are also used for motion matching.

3.3.2 Pose Update Pipeline

The core pose update pipeline of the `MotionMatchingUpdate` function executes the following five stages each frame:

1. **Input Processing & Trajectory Prediction**: First, the controller interprets the raw inputs (from keyboard or gamepad) relative to the camera's perspective to compute the desired character velocity and rotation. Instead of using the raw input directly, the system predicts the character's future trajectory (positions and directions) over a short window (e.g., 1 second). This prediction utilizes critical spring-damper systems (defined by a *halflife*) to generate a smooth path that blends naturally with the character's momentum, preventing unnatural, jerky movement changes.

2. **Query Vector Construction**: To find the most suitable animation frame, the system constructs a high-dimensional *query vector* (feature vector). This vector consists of:

   - **Trajectory Features**: The predicted future positions and facing directions of the character relative to its current root. This ensures the selected animation will steer the character along the desired path.
   - **Pose Features**: Velocity and position data of key end-effectors (feet and hips) from the current frame. This ensures continuity, preferring animations that match the current physical state (e.g., left foot moving forward).

3. **Database Search**: The constructed query vector is compared against a pre-computed database of motion features. The system performs a nearest-neighbor search to find the frame with the lowest "cost" (Euclidean distance in feature space). If a new frame significantly better than the naturally succeeding frame is found, the system elects to switch to this new animation clip.

4. **Inertialization (Transition Blending)**: When the system switches animation clips, a discontinuity (pop) occurs in the pose. Instead of traditional cross-fading, I used **Inertialization**. This technique computes the offset (positional and rotational difference) between the source pose and the target pose at the moment of transition. These offsets are then decayed to zero using spring-damper functions. This seamlessly bridges the two animations without the computational cost or "ghosting" artifacts of blending two full skeletons.

5. **Post-Processing with Inverse Kinematics (IK)**: Finally, IK is applied to correct foot placement. The motion matching system does not guarantee perfect contact with the ground.

   - **Foot Locking**: The system checks the database for "contact flags" (indicating if a foot should be planted). If a foot is in contact, I lock its position in world space to prevent "foot sliding" (where the feet slide on the ground while walking).
   - **Two-Bone IK**: The leg joints are then adjusted analytically to satisfy this locked foot position while maintaining the limb lengths.

Figure 2: Motion Matching Update Pipeline Flow

4 Results and Discussion

4.1 Visual Results

Figure 3 illustrates the results of the BVH Motion Player combined with the Linear Blend Skinning (LBS) system. The left panel shows the skeleton rendered using cuboids, confirming that the bone hierarchy and forward kinematics are correctly implemented. The right panel displays the skinned character mesh during playback, demonstrating that the vertices deform naturally following the underlying bone motions.

The complete Motion Matching system achieves responsive character control. Figure 4 illustrates the character in three distinct locomotion states, showcasing how the system selects appropriate poses based on speed and trajectory prediction.

Demonstration videos (MOV format) are located in the `result/task*` directories for a complete evaluation.

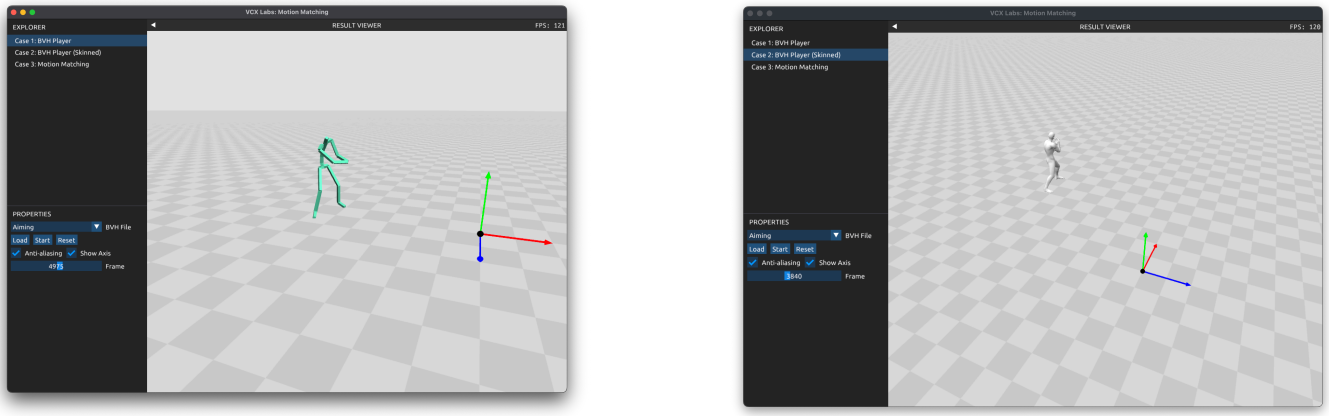
4.2 Discussion on Parameter Tuning

The behavior of the Motion Matching system is highly dependent on several key parameters. Based on my analysis and testing, the effects of the most critical parameters are summarized below:

1. **Simulation Half-life (Position & Rotation)**: This parameter controls the responsiveness of the Simulation Object, which the character follows. Low values make the character move unnaturally fast and twitchy, while high values make the movement feel heavy and sluggish. The optimal value is around **0.27**.

2. **Inertialization Half-life**: This parameter controls how quickly the offset vectors decay during animation transitions. Low values cause visible popping during transitions, while high values introduce floating or sliding artifacts. The optimal value is about **0.1 s**.

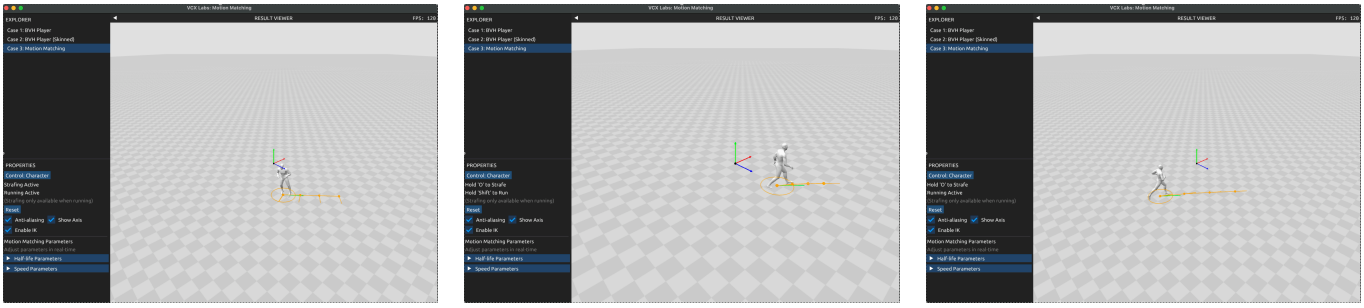
3. **Anisotropic Speed Settings**: This parameter controls the maximum movement speeds in different directions (forward, side, backward). Low speeds make locomotion feel slow and unresponsive, while high speeds can cause unnatural sliding. Suitable maximum speeds are roughly **4.0 m/s forward**, **3.0 m/s side**, and **2.5 m/s backward**.



(a) BVH Player: Skeleton Structure

(b) Skinned Player: Dynamic Mesh

Figure 3: BVH Motion Player with Linear Blend Skinning



(a) Strafing State

(b) Walking State

(c) Running State

Figure 4: Character transitions between Strafing, Walking, and Running states driven by user input.

5 Setup and Usage

The project is built using XMake. To compile the project, simply use the following commands:

```
xmake build -v
```

The executable can be located in the 'build' directory, in the subfolder corresponding to your platform. To run the project, use:

```
xmake run motion-matching
```

or directly execute the binary from the 'build' directory.